

07

线性方程组

7.3 初等行变换

增广矩阵与高斯消元：用行交换、行数乘、行倍加求解方程组、手解逆矩阵



鸢尾花书·《线性代数不难》 | 开源代码与书稿：<https://github.com/Visualize-ML>

本节你将掌握的核心技能

1 掌握三种初等行变换：行交换、行数乘、行倍加

3 用初等行变换将增广矩阵化为简化形式，求解线性方程组

5 理解主元 (pivot) 概念，及部分选主元 (行交换) 的作用

7 用Python实现高斯消元法求解线性方程组

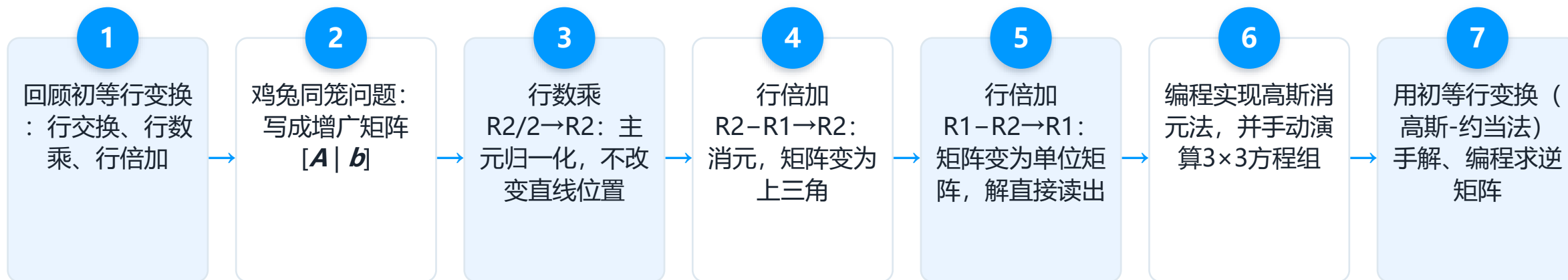
2 理解增广矩阵 $[A | b]$ 的构造方法

4 观察求解过程中直线、矩阵、增广矩阵、线性组合的同步变化

6 用初等行变换（高斯-约当消元法）手解逆矩阵

8 用Python实现初等行变换求逆矩阵

从增广矩阵，走向高斯消元与逆矩阵



初等行变换 (elementary row operations)

- ① 行交换 (row swapping): 交换选定的两行
- ② 行数乘 (row scaling): 某一行所有元素数乘一个非零标量
- ③ 行倍加 (row sum): 一行的非零倍数加到另一行

图1 • 原始方程组、线性组合、增广矩阵

- 3个头、8只脚： $x_1 + x_2 = 3$, $2x_1 + 4x_2 = 8$ ——两条斜线、 a_1 与 a_2 的线性组合

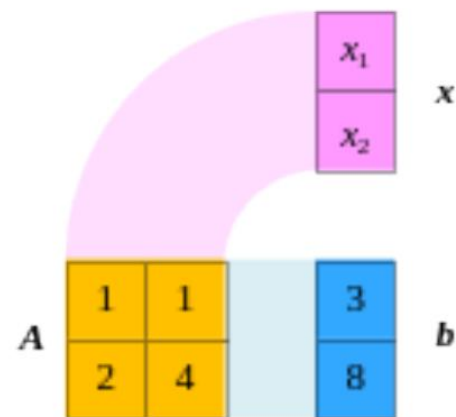
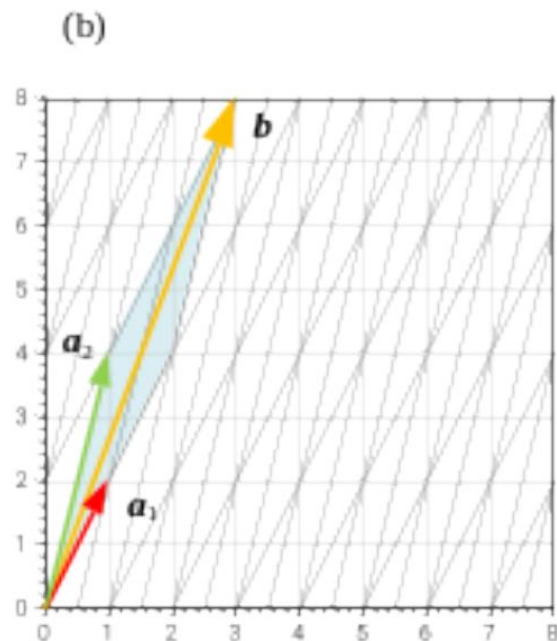
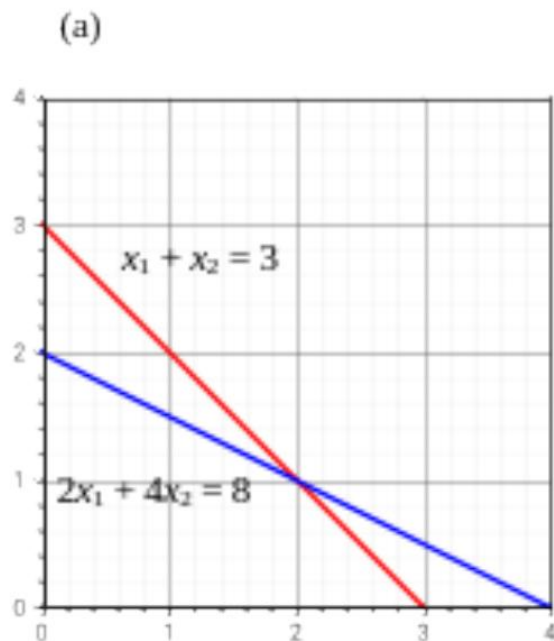


图 1. 原始方程组

$[A \mid b]$: 系数矩阵与常数向量拼接

$$[A \mid b] = [[1, 1 \mid 3], [2, 4 \mid 8]]$$

- 增广矩阵 (augmented matrix) 在系数矩阵 A 右侧附加常数向量 b ——有时会省略 A 、 b 之间的竖线
- 对增广矩阵进行初等行变换, 就可以求解线性方程组 $Ax = b$

a_1 、 a_2 不共线, $\det(A)=2$

$$x_1 a_1 + x_2 a_2 = b$$

- a_1 、 a_2 夹角小于90度但不共线（线性无关），网格为平行四边形，方阵 **A** 可逆，秩为2

R2/2 → R2

行数乘：主元归一化

$$R_2/2 \rightarrow R_2 \Rightarrow x_1 + x_2 = 3, \quad x_1 + 2x_2 = 4$$

- 把第2行所有元素除以2、替换原第2行，目的是让主元 (pivot) 变为1，便于后续运算

图2 · 行数乘不改变直线的位置、形态

- 比较图1(a)、图2(a)：两条直线完全一致，行数乘只是把方程两边同时缩放

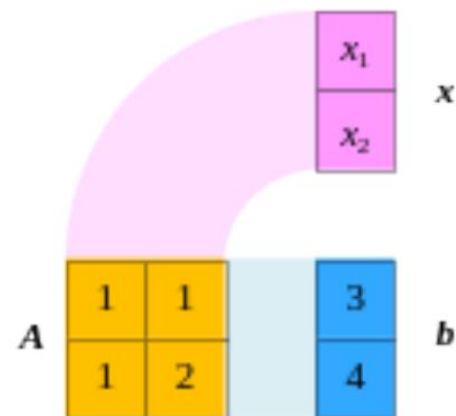
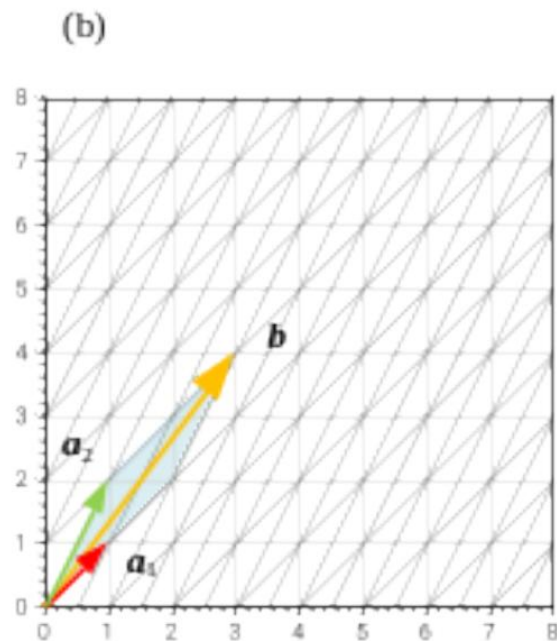
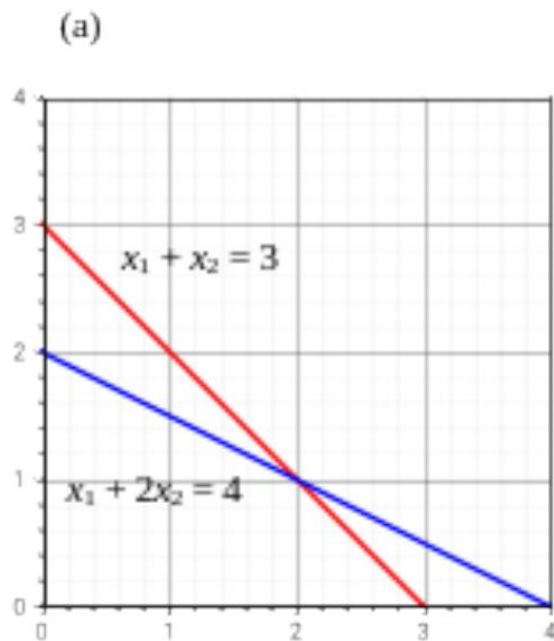


图 2. 行数乘, $R_2/2 \rightarrow R_2$

$R_2 - R_1 \rightarrow R_2$

行倍加：消去第2行的 x_1

$$R_2 - R_1 \rightarrow R_2 \Rightarrow x_1 + x_2 = 3, \quad x_2 = 1$$

- 第2行减去第1行、替换原第2行； $x_2=1$ 直接给出解，矩阵 A 也变成了上三角矩阵

图3 · 平行四边形网格变为横轴剪切网格

- $x_2=1$ 对应一条水平线， x_2 的解已经出现；行列式仍为1，网格形状发生剪切

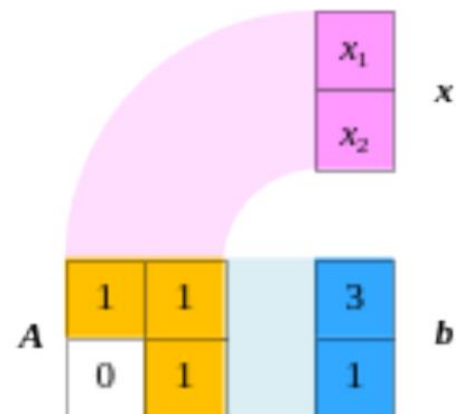
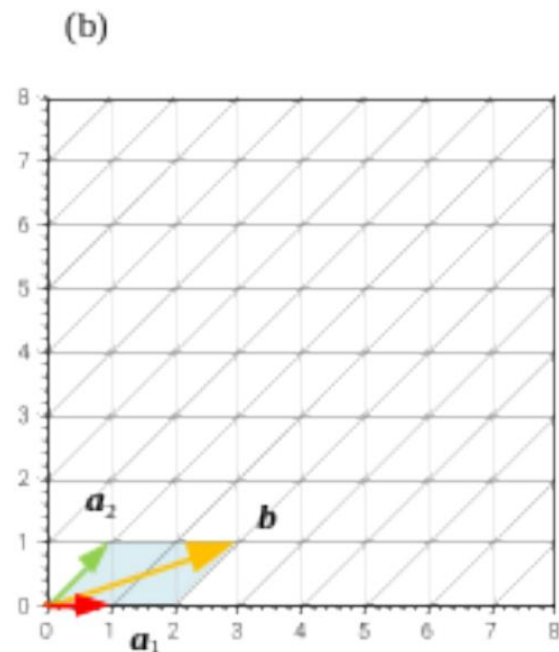
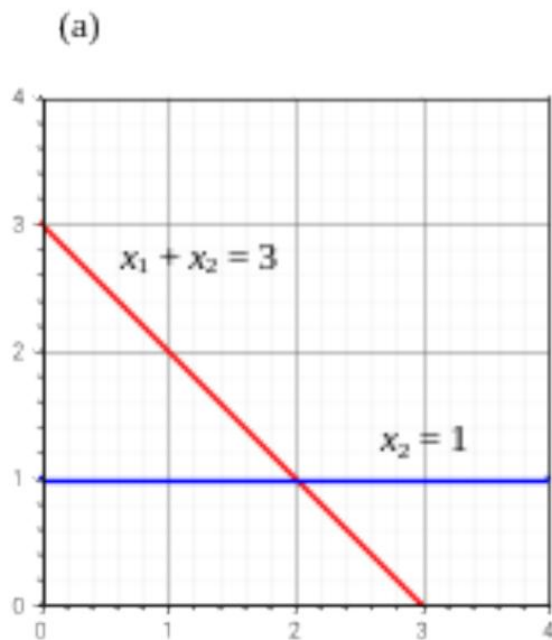


图 3. 行倍加， $R_2 - R_1 \rightarrow R_2$

$R_1 - R_2 \rightarrow R_1$

行倍加：矩阵 A 变为单位矩阵 I

$$R_1 - R_2 \rightarrow R_1 \Rightarrow x_1 = 2, x_2 = 1$$

- 第1行减去第2行、替换原第1行； A 变成 I 后， $Ax = b$ 直接化简为 $x = b$

图4 · 网格变为单位正方形，解已读出

- 两条直线变成竖直、水平线；两个列向量变成标准正交基 $[e_1, e_2]$

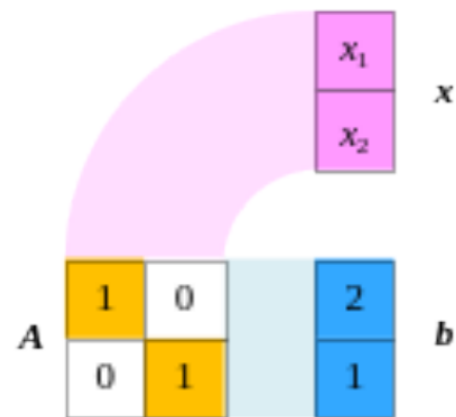
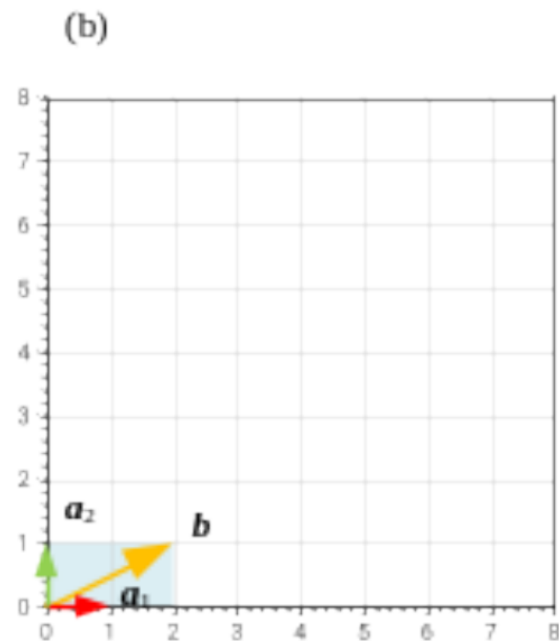
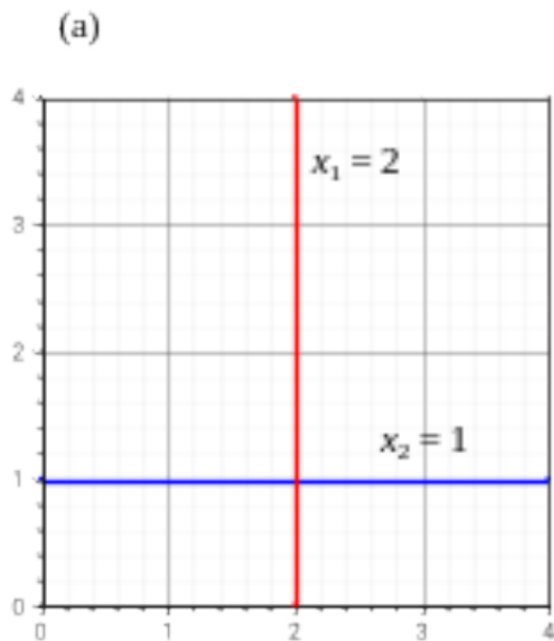


图 4. 行倍加, $R_1 - R_2 \rightarrow R_1$

小结

初等行变换求解，观察到了什么？

方程：两条直线从斜线，逐步变为水平线、竖直线，交点坐标即为解

矩阵 $\mathbf{A}$ ：从一般方阵，逐步变为上三角矩阵，最终变为单位矩阵 $\mathbf{I}$

增广矩阵 $[\mathbf{A} \mid \mathbf{b}]$ ：左侧化为 $\mathbf{I}$ ，右侧列向量就是方程组的解

线性组合：网格从平行四边形，逐步剪切、缩放，最终变为单位正方形

初等行变换求解线性方程组

`gauss_elimination(A, b)`: 部分选主元的高斯消元法

- for i in range(n): 按列索引遍历, 逐列把 A 化为上三角矩阵
- 行交换: 取当前列绝对值最大的元素所在行为主元行, 减少计算误差
- 行数乘: $Ab[i] = Ab[i] / Ab[i, i]$, 让主元归一化为 1
- 行倍加: $Ab[j] = Ab[j] - Ab[j, i] * Ab[i]$, 消去主元下方各行的当前列元素
- 反向代回: 从最后一行开始, 逐行向上求解 x 的每个分量

```
## 初始化
import numpy as np

## 自定义函数
def gauss_elimination(A, b):
    # 确保使用浮点数计算
    A = A.astype(float)
    b = b.astype(float)
    n = len(b)

    # 构造增广矩阵 [A|b]
    Ab = np.hstack([A, b.reshape(-1, 1)])

    # 原始矩阵 A > 上三角
    for i in range(n):
        # 1) 行交换, 选择绝对值最大为主元
        # 主元: 主对角线元素
        max_row = np.argmax(np.abs(Ab[i:, i])) + i
        Ab[[i, max_row]] = Ab[[max_row, i]]
        # 交换行, 行 i <-> 行 max_row

        # 2) 行数乘, 归一化主元, 主元变为 1
        Ab[i] = Ab[i] / Ab[i, i]

        # 3) 行倍加, 消元, 使主元下面所有行的 i 列变为 0
        for j in range(i+1, n):
            # Ab[j] -= Ab[j, i] * Ab[i]
            Ab[j] = Ab[j] - Ab[j, i] * Ab[i]

    # 回代求解, 上三角矩阵 > 单位矩阵
    x = np.zeros(n)
    for i in range(n-1, -1, -1):
        x[i] = Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:n])

    return x

## 定义矩阵 A 和向量 b
A = np.array([[1, 3, 6],
              [2, 7, 14],
              [0, 2, 5]])

b = np.array([25, 58, 19])

## 求解 Ax = b
x = gauss_elimination(A, b)
```

行交换 → 行数乘 → 行加倍

$$[[1, 3, 6 | 25], [2, 7, 14 | 58], [0, 2, 5 | 19]] \rightarrow [[1, 3.5, 7 | 29], [0, -0.5, -1 | -4], [0, 2, 5 | 19]]$$

- 第1列第2行绝对值最大，交换 R_1 、 R_2 ；主元归一化 $R_1/2 \rightarrow R_1$ ；再用 $R_2 - R_1 \rightarrow R_2$ 消去第1列

行交换 → 行数乘 → 行加倍

$$[[1, 3.5, 7|29], [0, -0.5, -1|-4], [0, 2, 5|19]] \rightarrow [[1, 3.5, 7|29], [0, 1, 2.5|9.5], [0, 0, 0.25|0.75]]$$

- 第2列主元以下绝对值最大的是第3行，交换 R_2 、 R_3 ；归一化 $R_2/2 \rightarrow R_2$ ；再消去第2列

$$x_3=3, \quad x_2=2, \quad x_1=1$$

$$x_3=3 \Rightarrow x_2=9.5-2.5\times 3=2 \Rightarrow x_1=29-3.5\times 2-7\times 3=1$$

- 矩阵 A 只有3列，第3列只需行数乘归一化；随后从最后一行开始反向代回，逐一求解

图5 • 初等行变换的流程

- 矩阵 A 的主元（绿色）逐列归一化、逐列消元，最终变为对角全绿的上三角矩阵



图 5. 初等行变换的流程

把 A 与 I 拼接，构造增广矩阵

$$[A \mid I] = [[1, 1 \mid 1, 0], [2, 4 \mid 0, 1]]$$

- 初等行变换不仅能解方程组，也能求逆矩阵——目标是把左侧 A 变为 I ，右侧 I 就变为 A^{-1}

重复同样的初等行变换

$$[A \mid I] \rightarrow [I \mid A^{-1}]$$

$$[A \mid I] \rightarrow [[1, 1 \mid 1, 0], [0, 1 \mid -1, 1/2]] \rightarrow [[1, 0 \mid 2, -1/2], [0, 1 \mid -1, 1/2]]$$

- 依次执行 $R_2/2 \rightarrow R_2$ 、 $R_2 - R_1 \rightarrow R_2$ 、 $R_1 - R_2 \rightarrow R_1$ ——与求解方程组完全相同的三步操作

验证

$$AA^{-1}=I, A^{-1}A=I$$

$$A^{-1} = \begin{bmatrix} 2 & -0.5 \\ -1 & 0.5 \end{bmatrix}$$

- 无论 A 左乘 A^{-1} , 还是 A^{-1} 左乘 A , 结果都是单位矩阵 I , 逆矩阵求解正确

初等行变换求解逆矩阵

`inverse_matrix(A)`: 高斯-约当消元法求逆

- 先检查 A 是否为方阵、 $\det(A)$ 是否为0, 不满足条件则返回None
- `A_I = np.hstack((A, I))`: 构造增广矩阵 $[A | I]$
- for `i in range(n)`: 逐列选主元、归一化、消去主元列的其他所有行元素
- 与解方程组不同: 这里消去的是主元上下所有行, 而不仅是主元下方
- `A_inv = A_I[:, n:]`: 提取增广矩阵右侧, 即为 A^{-1}

```
## 初始化
import numpy as np

## 自定义函数
def inverse_matrix(A):

    # 确保 A 是 NumPy 数组并转换为浮点数类型
    A = np.array(A, dtype=float)

    # 获取矩阵的行数和列数
    n, m = A.shape

    # 判断是否为方阵
    if n != m:
        print("错误: 矩阵不是方阵, 无法求逆。")
        return None

    # 计算行列式, 判断是否可逆
    if np.linalg.det(A) == 0:
        print("错误: 矩阵的行列式为 0, 无法求逆。")
        return None

    # 创建单位矩阵 I
    I = np.eye(n)

    # 构造增广矩阵 [A | I]
    A_I = np.hstack((A, I))

    # 进行高斯-约当消元法
    for i in range(n):
        # 1) 行交换, 选取主元
        max_row = np.argmax(np.abs(A_I[i:, i])) + i
        A_I[[i, max_row]] = A_I[[max_row, i]]

        # 2) 行数乘, 归一化主元 (主对角线元素变为 1)
        A_I[i] = A_I[i] / A_I[i, i]

        # 3) 行加倍, 消元 (使主元列的其他元素变为 0)
        for j in range(n):
            if i != j:
                A_I[j] -= A_I[j, i] * A_I[i]

    # 提取逆矩阵
    A_inv = A_I[:, n:]

    return A_inv

## 定义矩阵 A
A = np.array([[1, 3, 6],
              [2, 7, 14],
              [0, 2, 5]])

## 计算逆矩阵
A_inv = inverse_matrix(A)
```

初等行变换：求解方程组与求逆矩阵的统一工具

$$[A \mid b] \rightarrow [I \mid x]; [A \mid I] \rightarrow [I \mid A^{-1}]$$

- 行交换、行数乘、行倍加三种操作，通过部分选主元的高斯消元法，把 A 逐步化为 I
- 同一套操作，作用在 $[A \mid b]$ 上求解方程组，作用在 $[A \mid I]$ 上求逆矩阵

本节关键术语

初等行变换 *Elementary Row Operations* —— 行交换、行数乘、行倍加

行数乘 *Row Scaling* —— 某一行数乘一个非零标量

增广矩阵 *Augmented Matrix $[A | b]$* —— 系数矩阵与常数向量拼接

部分选主元 *Partial Pivoting* —— 选当前列绝对值最大的元素作主元, 减少误差

高斯-约当消元法 *Gauss-Jordan Elimination* —— 把 A 直接化为 I , 常用于求逆矩阵

行交换 *Row Swapping* —— 交换矩阵的两行

行倍加 *Row Sum* —— 一行的非零倍数加到另一行

主元 *Pivot* —— 消元过程中每行最左边的非零系数

高斯消元法 *Gaussian Elimination* —— 用初等行变换把 A 化为上三角矩阵再回代

反向代回 *Back Substitution* —— 从最后一行开始, 逐行向上求解未知数

延伸阅读

延伸阅读 · 初等矩阵

LA_07_03_03.ipynb 每一次初等行变换都对应左乘一个初等矩阵

延伸阅读 · LU分解

LA_07_03_04.ipynb 高斯消元过程本质上是对 A 做LU分解

延伸阅读 · `scipy.linalg.lu()`

LA_07_03_05.ipynb 用SciPy库函数完成LU、LDL分解

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

习题 Q1 – Q8

Q1 用初等行变换求解如下线性方程组，并绘制变换过程直线、线性组合的变化： $x_1 + x_2 = 4$ ， $2x_1 + 4x_2 = 12$ 。

Q3 用初等行变换计算Q2中矩阵的秩，并说明秩与矩阵是否可逆的关系。

Q5 什么是LU分解？它与高斯消元法（不做行交换）有什么关系？

Q7 什么是LDL分解？它适用于什么类型的矩阵？

Q2 用初等行变换求解矩阵 $A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$ 的逆矩阵，并验证 $AA^{-1} = I$ 。

Q4 什么是初等矩阵？每种初等行变换分别对应哪种初等矩阵？

Q6 自学用SciPy库函数`scipy.linalg.lu()`完成矩阵的LU分解。

Q8 自学使用SciPy库函数`scipy.linalg.ldl()`完成矩阵的LDL分解。

习题 Q9 – Q16

Q9 写一个函数`row_swap(M, i, j)`，实现矩阵 M 第 i 行、第 j 行的行交换。

Q11 写一个函数`row_add(M, i, j, k)`，实现把第 j 行的 k 倍加到第 i 行。

Q13 修改代码1的`gauss_elimination`函数，让它同时打印每一步变换后的增广矩阵。

Q15 构造一个部分选主元会失败的例子（如某列全为0），测试代码1函数的表现。

Q10 写一个函数`row_scale(M, i, k)`，实现将矩阵 M 第 i 行整体乘以非零标量 k 。

Q12 综合`row_swap`、`row_scale`、`row_add`，手写高斯消元，把本节 3×3 例子化为上三角矩阵。

Q14 用`numpy.linalg.solve()`求解本节 3×3 例子，并与代码1手写函数的结果对比。

Q16 用`matplotlib`动画展示图1到图4中直线随初等行变换逐步变化的过程。

习题 Q17 – Q25

Q17 写一个函数`is_upper_triangular(M)`，判断矩阵是否为上三角矩阵。

Q19 修改代码2的`inverse_matrix`函数，让它同时打印每一步变换后的增广矩阵。

Q21 构造一个奇异矩阵（行列式为0），测试代码2函数是否正确返回None。

Q23 比较高斯消元法（回代）与高斯-约当法（直接消到单位矩阵）的计算量差异。

Q25 综合练习：任选一个 4×4 可逆矩阵，用初等行变换手解（或编程）求其逆矩阵。

Q18 用numpy验证：上三角矩阵的行列式，等于其主对角线元素之积。

Q20 用`numpy.linalg.inv()`求解本节 2×2 例子的逆矩阵，并与代码2手写函数的结果对比。

Q22 写一个函数`gauss_jordan(A, b)`，用高斯-约当法（而非回代）直接求解线性方程组。

Q24 用Python分别实现行交换、行数乘、行倍加对应的3种初等矩阵，验证左乘等价于行变换。

10个可以和AI互动的Prompt

1 请解释为什么高斯消元法要用“绝对值最大的元素”作主元，而不是直接用主对角线元素？

2 请证明：每一次初等行变换，都等价于矩阵左乘一个特定的初等矩阵。

3 请解释增广矩阵 $[A \mid b]$ 与线性方程组 $Ax = b$ 之间的对应关系。

4 请用Python演示：用初等行变换手解一个 3×3 方程组，并打印每一步的增广矩阵。

5 请解释高斯消元法和高斯-约当消元法的区别，分别适合什么场景？

10个可以和AI互动的Prompt

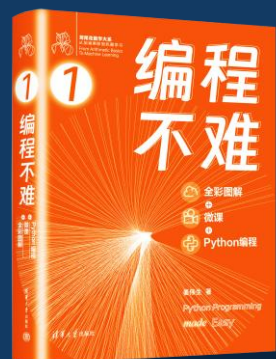
- 6 请解释为什么求逆矩阵时，要消去主元列上、下所有行的元素，而不只是下方？
- 7 请解释LU分解与高斯消元法之间的关系，并说明L、U矩阵分别代表什么。
- 8 请举一个现实场景，说明求解线性方程组或求逆矩阵在工程或数据科学中的应用。
- 9 请用NumPy对比自己实现的高斯消元法与`numpy.linalg.solve()`在大规模矩阵上的运行速度。
- 10 请设计一个Python小项目：输入任意 $n \times n$ 可逆矩阵，逐步展示初等行变换求逆矩阵的完整过程。

数学

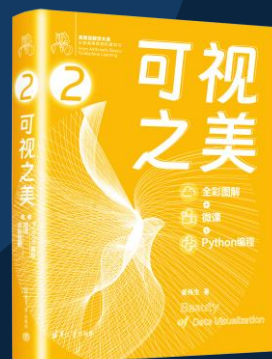
数学基础

线性代数

概率统计

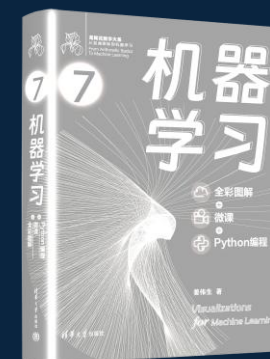
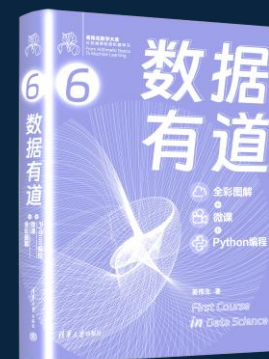
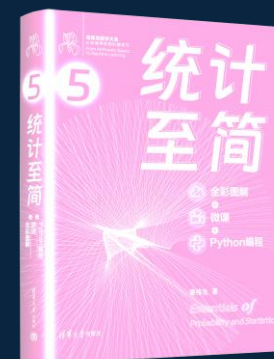
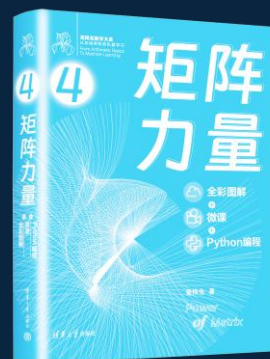
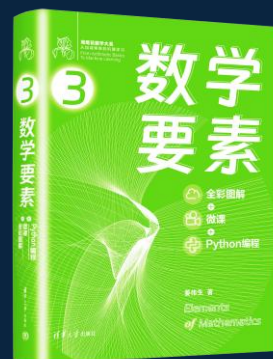


Python编程



数据可视化

工具



数据分析
图和网络

回归、降维
分类、聚类

实践

